LangChain Debugging — Worked Example: Solution

Back to the exercise · Round 1: Basic · Round 2: Intermediate · Round 3: Real-world · Round 4: Multi-agent · Hints

Story — `langchain_debugging_practice.py`: every round here is about what actually goes *into* a prompt template — the key names it expects, the instructions it carries, and exactly where outside text gets inserted into it. A prompt template can be inspected and checked on its own, before any model call ever runs, so every test here needs no API key at all. **If not:** you'd end up testing chains only by calling the real model, paying for every single check, and still not clearly see which part of the chain actually broke.

Every fix and test below goes in `practice/langchain_debugging_practice.py`, and runs with `pytest langchain_debugging_practice.py -v` from inside `practice/`.

- Round 1: Basic
- Round 2: Intermediate
- Round 3: Real-world
- Round 4: Multi-agent
- Real cause vs. symptom fix

Round: Basic

Real cause: the prompt template declares one variable, `{question}`, but the caller invoked the chain with the key `"input"`. LangChain's error names the mismatch directly (`Expected: ['question'] Received: ['input']`) — it's easy to miss because `KeyError` makes it feel like a Python dictionary bug rather than a naming mismatch.

Story: the error message itself already contains the full answer, if you actually read it. This round trains reading the whole message slowly and carefully before jumping to a guess. **If not:** you'd end up blaming the API or LangChain itself, for what's really a one-word typo in your own call.

The fix:

```
from langchain_core.prompts import ChatPromptTemplate

prompt = ChatPromptTemplate.from_template("Answer this question: {question}")
```

```
# why: the dict key must match the template's {question} exactly
prompt_input = {"question": "What is the refund policy?"}
```

Then the call is `chain.invoke(prompt_input)`.

Test that would have caught it:

```
def test_prompt_accepts_the_question_key():
    # how: invoking the prompt alone fills the template — no model call
    text = prompt.invoke({"question": "What is the refund policy?"})
    assert "What is the refund policy?" in text.to_string()
```

Testing the prompt on its own catches every key mismatch for free, before the model is ever involved.

Round: Intermediate

Real cause: the prompt never tells the model what shape to write. `PydanticOutputParser` checks the reply against `Answer`, but the model has never seen `Answer` — so it guesses, and `"confidence": "high"` is a perfectly human guess. As Doc05 says, `PydanticOutputParser` needs `parser.get_format_instructions()` inserted into the prompt, or the model never learns the shape. The API call is fine; the bug is in the chain's prompt layer.

Story: the error surfaces from the parser, at the very end of the chain, but the actual cause sits at the start — in the prompt itself. This round trains following a failure *backward* through the chain to where it genuinely originates. **If not:** you'd add retries or loosen `confidence`'s type to a plain string, and the model would just keep guessing a different wrong shape every time.

The fix:

```
from pydantic import BaseModel
from langchain_core.output_parsers import PydanticOutputParser
from langchain_core.prompts import ChatPromptTemplate

class Answer(BaseModel):
    summary: str
    confidence: float

parser = PydanticOutputParser(pydantic_object=Answer)

template = (
    "Answer this question: {question}\n\n"
    "{format_instructions}"
```

```
)
# why: .partial fills {format_instructions} once, so the model sees
# the exact shape (and that confidence is a number) on every call
prompt = ChatPromptTemplate.from_template(template).partial(
    format_instructions=parser.get_format_instructions()
)
```

Then `chain = prompt | ChatOpenAI() | parser`, same as before.

Test that would have caught it:

```
def test_prompt_tells_the_model_the_answer_shape():
    text = prompt.invoke({"question": "What is the refund policy?"})
    rendered = text.to_string()
    assert "confidence" in rendered
    assert "number" in rendered

def test_parser_accepts_the_shape_the_prompt_asks_for():
    reply = '{"summary": "Refunds within 30 days.", "confidence": 0.9}'
    result = parser.parse(reply)
    assert result.confidence == 0.9
```

The first test checks that the instructions reach the prompt; the second checks that a reply in that shape parses. Neither needs a model call.

Round: Real-world

Real cause: the user's text was put *into the template itself* (`("human", user_question)`), so LangChain reads it as template text — and every `{...}` in it as a variable it must fill. `{"status": "error"}` becomes a "missing variable" named `"status"`. Text passed as a *value* for a variable is never read this way, so the fix is to keep a fixed template with a `{question}` slot and pass the user's text in as its value.

Story: the error's own "Note" suggests escaping the braces, which would make this one specific error go away while still leaving raw user text embedded inside the template — exactly where the next odd character can break it again, differently. This round trains asking *why* a suggestion works before taking it at face value. **If not:** you'd add a one-off escape helper, and every new place that builds a template from outside text would need to remember to call it too.

The fix:

```
from langchain_core.prompts import ChatPromptTemplate

# why: the template is fixed and built ONCE — user text never
```

```
# becomes template text, it only fills the {question} slot
prompt = ChatPromptTemplate.from_messages([
    ("system", "You answer support questions briefly."),
    ("human", "{question}"),
])

def build_answer_input(user_question):
    # how: the user's text is a VALUE — braces in it are just text
    return {"question": user_question}
```

Then `answer()` becomes `chain.invoke(build_answer_input(user_question))`, with `chain = prompt | ChatOpenAI() | StrOutputParser()` built once.

Test that would have caught it:

```
def test_question_with_curly_braces_reaches_the_model_unchanged():
    tricky = 'Why does my config return {"status": "error"}?'
    messages = prompt.invoke(build_answer_input(tricky)).to_messages()
    assert messages[-1].content == tricky
```

A test suite of clean, hand-typed questions never reaches this path — the test has to include a question with braces in it on purpose, the way real users eventually will.

Round: Multi-agent

Real cause: the same mistake as Round 3 — outside text glued into the template itself — but the text now comes from another agent's output instead of a user. Fixing `qa.py` didn't cover it, because the fix was applied at one place instead of as a rule: *any* text that comes from outside (a user, a tool, another agent) must be a variable's value, never part of the template.

Story: the bug only appears when real Research output happens to contain a stray JSON-like snippet — exactly the kind of thing nobody thinks to put in a hand-written test case. This round trains applying a real-cause fix everywhere the same underlying pattern exists in the codebase, not just at the one spot where it happened to be first noticed. **If not:** every agent that builds a prompt from another agent's free-text output would carry the exact same hidden crash, waiting to happen again elsewhere.

The fix:

```
from langchain_core.prompts import ChatPromptTemplate
from langgraph.types import Command

# why: fixed template, one {findings} slot — built once
VERIFY_PROMPT = ChatPromptTemplate.from_template(
```

```
    "Check this claim against the research findings:\n{findings}"
)

def verify_claim(state):
    # how: Research's text is passed as a value, never as template text
    chain = VERIFY_PROMPT | ChatOpenAI() | StrOutputParser()
    result = chain.invoke({"findings": state["research_findings"]})
    return Command(goto="writer_agent", update={"analysis": result})
```

Test that would have caught it:

```
def test_findings_with_json_snippets_reach_the_prompt_unchanged():
    findings = 'the API returned {"error": "rate_limited"} on the first try'
    text = VERIFY_PROMPT.invoke({"findings": findings}).to_string()
    assert findings in text
```

The test feeds the Analysis agent's prompt the kind of text Research really produces — including a JSON snippet — without running the full 4-agent pipeline or calling a model.

Real cause vs. symptom fix

A symptom fix here would be catching the `KeyError` from Round 1 and falling back to a generic answer, retrying the parse until the model happens to write a number (Round 2), or escaping braces in the user's text — exactly what the error's own "Note" suggests — at the one call that happened to break (Rounds 3 and 4). Escaping makes today's error disappear, but it leaves outside text inside the template, where it can still break things (or change the prompt) in ways you haven't seen yet. The real cause in Rounds 3 and 4 is where the text sits, not which characters it contains — so the real fix (a fixed template, outside text only as a variable's value) works for any text, from any source, without anyone needing to remember to escape it. That it fixes both rounds with the same rule is a good sign you found the real cause: a real fix generalizes; a symptom fix needs re-applying everywhere the bug shows up next.